SHODASHA NUMBERS

A Geometric Base-16 Representation with a Native Glyph and Naming System

Author:
Mohit Kashyap
mohit.kashyap@peersock.com

Status:
Under Development / Draft Specification

Date:
31st August, 2026

1. Executive Summary

Shodasha is a proposed visual notation for base-16 values. It keeps hexadecimal's 4-bit structure but replaces the alphabetic symbols A-F with a dedicated geometric glyph set. Each glyph is generated from four spatial quadrants, with each quadrant representing one bit of a hexadecimal nibble. This creates a deterministic relationship between a four-bit value and its visual glyph.

The project is intended as a structural extension of hexadecimal rather than a replacement for binary, decimal numbers, or hexadecimal computation. Its main purpose is to provide another way to represent hexadecimal values for human use, including writing, visual recognition, teaching, typography, and software-based representation.

The specification also defines separate visual forms for NULL, TRUE, and FALSE. HEAD and TAIL are optional markers that can be used when the beginning and end of a data stream need to be made explicit. A spoken naming system is also included so that Shodasha values can be read without using the Latin A-F convention.

This document describes the proposed Shodasha system and its design. It does not claim that Shodasha is already faster, easier to read, or better recognized by machines than conventional hexadecimal. These properties can be tested and evaluated through future implementations and practical use.

1.1 Design Goals

- Provide sixteen dedicated geometric glyphs for base-16 values.
- Keep a deterministic relationship between each glyph and its 4-bit binary value.
- Keep the glyph family small and geometric enough to be reproduced by hand, in a font, or by software.
- Provide distinct visual representations for NULL and boolean values.
- Provide optional HEAD and TAIL markers when explicit stream framing is useful.
- Provide a spoken naming system for single digits and larger positional values.
- Keep the system open to implementation, testing, criticism, and future development.

1.2 Scope and Non-Goals

Shodasha does not attempt to replace binary logic, CPU instruction sets, decimal arithmetic, or the hexadecimal representation already used by software and hardware. The proposal is focused on notation and representation.

A software implementation can continue to store and calculate values as ordinary integers, bytes, or bit strings while using Shodasha for display, input, communication, or other human-facing purposes.

The project also does not propose a new programming language or a new general-purpose writing system. The naming system is specifically intended for reading and communicating Shodasha values.

2. Motivation

This section explains the main ideas behind Shodasha and why a different visual representation of hexadecimal may be useful. The focus is on the human-facing side of hexadecimal: how values are written, read, recognized, and spoken.

2.1 Why Keep Base-16?

Hexadecimal remains useful because one hexadecimal digit represents four binary bits. This makes it a compact human notation for byte-oriented and bit-oriented data. Shodasha keeps this property. The change is in the visible symbols, not in the numerical meaning.

2.2 The Human-Facing Representation

The conventional hexadecimal system uses the symbols 0-9 and A-F. This is practical and already widely established in computing, but it means that one numeral system is represented partly by digits and partly by letters. Shodasha explores whether a dedicated geometric character set can make the representation more visually uniform.

Multi-digit hexadecimal values do not have a dedicated naming system in the same way that ordinary numbers do. In practice, the individual hexadecimal symbols are usually read separately, for example: ‘A’, ‘5’ & ‘D’ in A5D

Shodasha includes a naming system so that its values can be spoken using dedicated names rather than the Latin A-F convention.

Hexadecimal strings can also contain sequences that look like ordinary words, identifiers, or abbreviations. This is not a parser vulnerability by itself because software normally has a defined grammar and context. It is better understood as a human-interface issue that a dedicated numeral alphabet may avoid.

2.3 Human Recognition and OCR

Handwritten characters can become ambiguous when different symbols share similar shapes. Shodasha therefore uses a restricted geometric vocabulary and can optionally place a data stream between HEAD and TAIL markers. The purpose of framing is to provide additional context to a human or machine reader; it is not claimed here to make recognition flawless without testing.

The same geometric structure also gives each Shodasha glyph a defined relationship with its four-bit value. This makes the glyphs suitable for software-based representation and possible machine recognition.

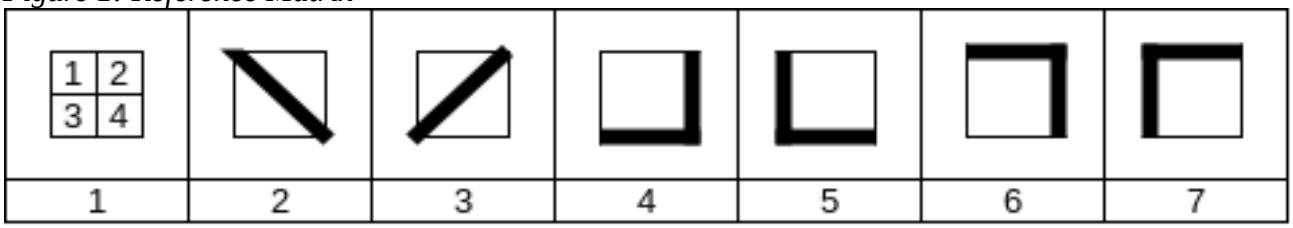
3. Shodasha Glyph Architecture

This section establishes the geometric rules governing the Shodasha character set. By defining a strict spatial grid, the architecture maps binary bit states directly to visual configurations, creating a deterministic relationship between a number's value and its written shape.

3.1 Four-Quadrant Matrix

The visual architecture of a Shodasha character is governed by a two-dimensional coordinate framework divided into four symmetrical quadrants. As shown in **Part 1** of **Figure 1: Reference Matrix**, the spatial layout corresponds directly to a 4-bit binary nibble structured as follows:

Figure 1: Reference Matrix



Each quadrant represents one specific bit of the 4-bit binary sequence, arranged from Most Significant Bit to Least Significant Bit: 2^3, 2^2, 2^1, 2^0. Instead of arbitrary curves, the character's final shape is determined by whether each quadrant holds a binary value of 0 or 1.

Geometric Representation of Binary Zero (0): When a quadrant holds a binary state of 0, it projects a specific diagonal path across that sector of the character's frame:

1. **Quadrants 1 and 4:** A binary 0 in either of these positions generates a diagonal stroke running from the top-left toward the bottom-right as shown in **Part 2**.
2. **Quadrants 2 and 3:** A binary 0 in either of these positions generates a diagonal stroke running from the bottom-left toward the top-right as shown in **Part 3**.

Consequently, if a number contains zeros across matching diagonal pairs, the strokes merge into a single continuous, unbroken diagonal line.

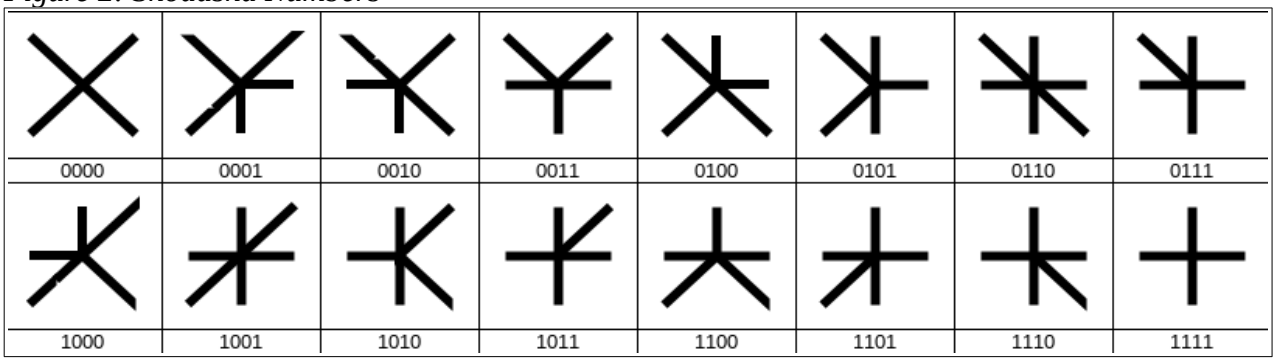
Geometric Representation of Binary One (1): When a quadrant holds a binary state of 1, it is represented by drawing its inner axial boundaries—the vertical and horizontal segments shared by that quadrant and the central axis. Because numbers are written without an outer bounding box, these strokes form distinct, sharp right-angle corners meeting at the center axis:

1. **Quadrant 1 (Top-Left):** Formed by drawing its right vertical and bottom horizontal inner boundaries as shown in **Part 4**.
2. **Quadrant 2 (Top-Right):** Formed by drawing its left vertical and bottom horizontal inner boundaries as shown in **Part 5**.
3. **Quadrant 3 (Bottom-Left):** Formed by drawing its right vertical and top horizontal inner boundaries as shown in **Part 6**.
4. **Quadrant 4 (Bottom-Right):** Formed by drawing its left vertical and top horizontal inner boundaries as shown in **Part 7**.

3.2 Reference Glyph Directory

This section defines the precise geometric composition for each of the sixteen individual Shodasha characters based on their underlying 4-bit binary states. By combining the zero diagonal paths and the right-angle binary corners defined in Section 3.1, every digit forms a distinct, predictable glyph as shown in **Figure 2: Shodasha Numbers** *(**Note —** Pronunciations follow the naming conventions defined in Section 5)*.

Figure 2: Shodasha Numbers



1. **Sif (0 / 0000)**
   - **Composition:** All four quadrants hold a state of 0.
   - **Glyph Geometry:** The merging of both diagonal paths (Quadrants 1 & 4 and Quadrants 2 & 3) creates a uniform, perfectly symmetrical diagonal cross (X configuration).
2. **Numbers 1 to 3:** Low-Order Bit Mutations
   - **Uni (1 / 0001):** Quadrant 4 holds a 1 (forming a bottom-right corner stroke), while Quadrants 1, 2, and 3 default to diagonal paths representing 0.
   - **Ome (2 / 0010):** Quadrant 3 holds a 1 (forming a bottom-left corner stroke), while Quadrants 1, 2, and 4 track the zero diagonal lines.
   - **Tri (3 / 0011):** Quadrants 3 and 4 simultaneously hold a 1 (merging to form a solid, continuous horizontal baseline), while Quadrants 1 and 2 track their baseline zero diagonals.
3. **Numbers 4 to 7:** Mid-Scale Formations
   - **Char (4 / 0100):** Quadrant 2 holds a 1 (forming a top-right corner stroke), while the remaining quadrants default to diagonal lines.
   - **Pen (5 / 0101):** Quadrants 2 and 4 hold a 1, merging their axial bounds to form a distinct, straight vertical right boundary line.
   - **Liu (6 / 0110):** Quadrants 2 and 3 hold a 1, forming an interconnected, stepped geometric path crossing the central axis.
   - **Sep (7 / 0111):** Quadrants 2, 3, and 4 all hold a state of 1, enclosing three out of four directions into an open-left box structure anchored by a top-left diagonal line.
4. **Numbers 8 to 11:** High-Order Bit Dominance
   - **Oct (8 / 1000):** Quadrant 1 holds a 1 (forming a top-left corner stroke), while Quadrants 2, 3, and 4 map diagonal zero lines.
   - **Nau (9 / 1001):** Quadrants 1 and 4 hold a 1, creating a distinct diagonal split balanced by opposite binary corners.
   - **Kumi (10 / 1010):** Quadrants 1 and 3 hold a 1, merging their axial boundaries to create a straight, solid vertical left boundary line.
   - **Isa (11 / 1011):** Quadrants 1, 3, and 4 hold a 1, enclosing three sections into an open-top-right configuration.
5. **Numbers 12 to 15:** High-Order Combinations

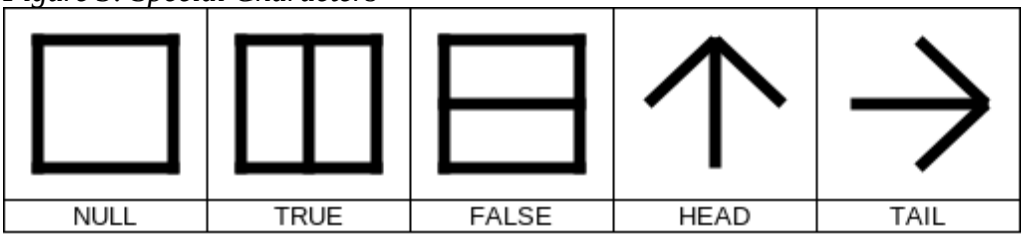
- **Kur (12 / 1100):** Quadrants 1 and 2 hold a 1, merging their inner boundaries to create a solid, continuous horizontal top line.
- **Oru (13 / 1101):** Quadrants 1, 2, and 4 hold a 1, creating a highly uniform box configuration that remains open strictly on the bottom-left axis.
- **Ran (14 / 1110):** Quadrants 1, 2, and 3 hold a 1, forming a prominent box structure that remains open strictly on the bottom-right axis.
- **Zen (15 / 1111):** All four quadrants hold a state of 1. The intersecting inner axial strokes join seamlessly to form a standard central plus-sign (+ configuration), representing absolute maximum density.

4. Special Characters

NULL, TRUE, and FALSE are separate from the sixteen numeric glyphs, as shown in **Figure 3: Special Characters**. This prevents a parser or human reader from treating a type marker as an ordinary numeric value. Their enclosing geometry is intentionally different from the unboxed numeric characters.

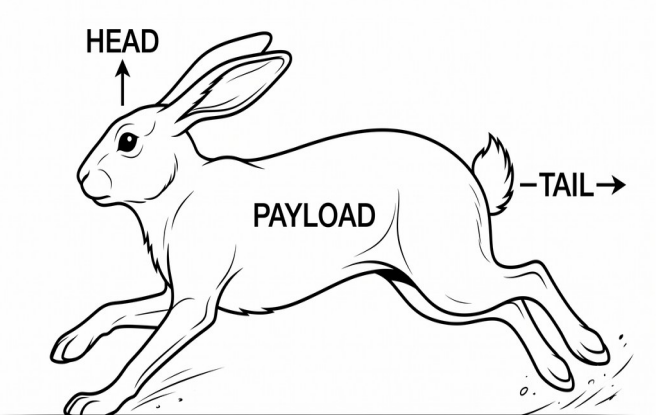
HEAD and TAIL are framing and stream-context markers. In this specification, they are OPTIONAL. A Shodasha value may be written without them when the surrounding syntax already provides sufficient context. They may be included when a stream needs explicit beginning, orientation, or ending information.

Figure 3: Special Characters



4.1 Optional Framing Rule

When explicit framing is desired, a Shodasha stream may be written as HEAD + payload + TAIL. The framing markers do not change the semantic value of the payload. They provide context so that a reader or parser can identify the intended direction and boundaries.



Because HEAD and TAIL are optional, implementations must not reject an otherwise valid Shodasha value merely because the markers are absent. An implementation may define a stricter protocol profile that requires framing, but that profile must be stated separately from the core notation.

5. Shodasha Naming System

The names are taken from word forms in different languages to make them easy to write, read, and pronounce. The naming system is designed to make the values more systematic and easier to learn rather than requiring every name to be memorized separately.

5.1 Single-Digit Names

A native naming system is integrated into Shodasha to provide a practical way to read visual values aloud. The names consist of short roots influenced by Esperanto and other languages. The goal is to establish a compact, regular naming scheme that avoids the Latin A-F letters entirely.

Hexadecimal Value	Esperanto-Oriented Form / English Spelling
0	Sif / Sif
1	Uni / Uni
2	Ome / Ome
3	Tri / Tri
4	Ĉar / Char
5	Pen / Pen
6	Liu / Liu
7	Sep / Sep
8	Okt / Oct
9	Nau / Nau
A	Kumi / Kumi
B	Isa / Isa
C	Kur / Kur
D	Oru / Oru
E	Ran / Ran
F	Zen / Zen

5.2 Positional Compounding

The naming system groups larger values according to their hexadecimal positions. The draft uses -jo (pronounced -yo) for the next positional level and -ja (pronounced -ya) for the following level, with additional positional names introduced for larger values. This allows hexadecimal values to be spoken systematically without reverting to the Latin letters A–F.

Each hexadecimal digit is represented by its corresponding Shodasha name. Positional suffixes identify the digit's hexadecimal place value. Within a compound name, higher-order positions are spoken before lower-order positions, and zero-valued positions may be omitted according to the naming rules.

The following examples illustrate how hexadecimal values are formed and pronounced using the positional naming system. The examples also show the corresponding English spelling alongside the Esperanto-oriented form.

Hexadecimal Value	Esperanto-Oriented Form / English Spelling
10	Unijo / Uniyo
11	Unijo Uni / Uniyo Uni
1F	Unijo Zen / Uniyo Zen
20	Omejo / Omeyo
21	Omejo Uni / Omeyo Uni
2F	Omejo Zen / Omeyo Zen

Hexadecimal Value	Esperanto-Oriented Form / English Spelling
30	Trijo / Triyo
31	Trijo Uni / Triyo Uni
3F	Trijo Zen / Triyo Zen
40	Ĉarjo / Charyo
4F	Ĉarjo Zen / Charyo Zen
50	Penjo / Penyo
5F	Penjo Zen / Penyo Zen
60	Liujo / Liuyo
6F	Liujo Zen / Liuyo Zen
70	Sepjo / Sepyo
7F	Sepjo Zen / Sepyo Zen
80	Oktjo / Octyo
8F	Oktjo Zen / Octyo Zen
90	Naujo / Nauyo
9F	Naujo Zen / Nauyo Zen
A0	Kumijo / Kumiyo
AF	Kumijo Zen / Kumiyo Zen
B0	Isajo / Isayo
BF	Isajo Zen / Isayo Zen
C0	Kurjo / Kuryo
CF	Kurjo Zen / Kuryo Zen
D0	Orujo / Oruyo
DF	Orujo Zen / Oruyo Zen
E0	Ranjo / Ranyo
EF	Ranjo Zen / Ranyo Zen
F0	Zenjo / Zenyo
FF	Zenjo Zen / Zenyo Zen
100	Unija / Uniya
101	Unija Uni / Uniya Uni
110	Unija Unijo / Uniya Uniyo
11F	Unija Unijo Zen / Uniya Uniyo Zen
120	Unija Omejo / Uniya Omeyo
12F	Unija Omejo Zen / Uniya Omeyo Zen
130	Unija Trijo / Uniya Triyo
13F	Unija Trijo Zen / Uniya Triyo Zen
1FF	Unija Zenjo Zen / Uniya Zenyo Zen
200	Omeja / Omeya
2FF	Omeja Zenjo Zen / Omeya Zenyo Zen
300	Trija / Triya
3FF	Trija Zenjo Zen / Triya Zenyo Zen
400	Ĉarja / Charya
4FF	Ĉarja Zenjo Zen / Charya Zenyo Zen
500	Penja / Penya
5FF	Penja Zenjo Zen / Penya Zenyo Zen
600	Liuja / Liuya
6FF	Liuja Zenjo Zen / Liuya Zenyo Zen
700	Sepja / Sepya
7FF	Sepja Zenjo Zen / Sepya Zenyo Zen
800	Oktja / Octya
8FF	Oktja Zenjo Zen / Octya Zenyo Zen
900	Nauja / Nauya

Hexadecimal Value	Esperanto-Oriented Form / English Spelling
9FF	Nauja Zenjo Zen / Nauya Zenyo Zen
A00	Kumija / Kumiya
AFF	Kumija Zenjo Zen / Kumiya Zenyo Zen
B00	Isaja / Isaya
BFF	Isaja Zenjo Zen / Isaya Zenyo Zen
C00	Kurja / Kurya
CFF	Kurja Zenjo Zen / Kurya Zenyo Zen
D00	Oruja / Oruya
DFF	Oruja Zenjo Zen / Oruya Zenyo Zen
E00	Ranja / Ranya
EFF	Ranja Zenjo Zen / Ranya Zenyo Zen
F00	Zenja / Zenya
FFF	Zenja Zenjo Zen / Zenya Zenyo Zen

5.3 Large Positional Names

The large positional names are the units used by the draft's examples. The naming rules should be treated as a separate linguistic layer so that future revisions can change pronunciation or naming conventions without changing the numeric glyph specification.

For large values, the draft groups hexadecimal digits into three-digit clusters. This is a presentation and pronunciation rule rather than a change to the underlying base-16 value. A triad contains three hexadecimal digits, or twelve bits.

Triad grouping is intended to make long values easier to scan and pronounce. It should not be confused with byte boundaries, memory boundaries, or an internal storage format. Software may store the same value without any grouping.

Hexadecimal Value (Triad-Grouped)	Esperanto-Oriented Form / English Spelling
1,000	Uni Sah / Uni Sah
F,FFF	Zen Sah, Zenja Zenjo Zen / Zen Sah, Zenya Zenyo Zen
10,000	Unijo Sah / Uniyo Sah
FF,FFF	Zenjo Zen Sah, Zenja Zenjo Zen / Zenyo Zen Sah, Zenya Zenyo Zen
100,000	Unija Sah / Uniya Sah
FFF,FFF	Zenja Zenjo Zen Sah, Zenja Zenjo Zen / Zenya Zenyo Zen Sah, Zenya Zenyo Zen
1,000,000	Uni Ŭan / Uni Wan
F,FFF,FFF	Zen Ŭan, Zenja Zenjo Zen Sah, Zenja Zenjo Zen / Zen Wan, Zenya Zenyo Zen Sah, Zenya Zenyo Zen
10,000,000	Unijo Ŭan / Uniyo Wan
FF,FFF,FFF	Zenjo Zen Ŭan, Zenja Zenjo Zen Sah, Zenja Zenjo Zen / Zenyo Zen Wan, Zenya Zenyo Zen Sah, Zenya Zenyo Zen
100,000,000	Unija Ŭan / Uniya Wan
FFF,FFF,FFF	Zenja Zenjo Zen Ŭan, Zenja Zenjo Zen Sah, Zenja Zenjo Zen / Zenya Zenyo Zen Wan, Zenya Zenyo Zen Sah, Zenya Zenyo Zen
1,000,000,000	Uni Kje / Uni Kye
F,FFF,FFF,FFF	Zen Kje, Zenja Zenjo Zen Ŭan, Zenja Zenjo Zen Sah, Zenja Zenjo Zen / Zen Kye, Zenya Zenyo Zen Wan, Zenya Zenyo Zen Sah, Zenya Zenyo Zen

Hexadecimal Value (Triad-Grouped)	Esperanto-Oriented Form / English Spelling
10,000,000,000	Unijo Kje / Uniyo Kye
FF,FFF,FFF,FFF	Zenjo Zen Kje, Zenja Zenjo Zen Ŭan, Zenja Zenjo Zen Sah, Zenja Zenjo Zen / Zenyo Zen Kye, Zenya Zenyo Zen Wan, Zenya Zenyo Zen Sah, Zenya Zenyo Zen
100,000,000,000	Unija Kje / Uniya Kye
FFF,FFF,FFF,FFF	Zenja Zenjo Zen Kje, Zenja Zenjo Zen Ŭan, Zenja Zenjo Zen Sah, Zenja Zenjo Zen / Zenya Zenyo Zen Kye, Zenya Zenyo Zen Wan, Zenya Zenyo Zen Sah, Zenya Zenyo Zen
1,000,000,000,000	Uni Alp / Uni Alp
F,FFF,FFF,FFF,FFF	Zen Alp, Zenja Zenjo Zen Kje, Zenja Zenjo Zen Ŭan, Zenja Zenjo Zen Sah, Zenja Zenjo Zen / Zen Alp, Zenya Zenyo Zen Kye, Zenya Zenyo Zen Wan, Zenya Zenyo Zen Sah, Zenya Zenyo Zen
10,000,000,000,000	Unijo Alp / Uniyo Alp
FF,FFF,FFF,FFF,FFF	Zenjo Zen Alp, Zenja Zenjo Zen Kje, Zenja Zenjo Zen Ŭan, Zenja Zenjo Zen Sah, Zenja Zenjo Zen / Zenyo Zen Alp, Zenya Zenyo Zen Kye, Zenya Zenyo Zen Wan, Zenya Zenyo Zen Sah, Zenya Zenyo Zen
100,000,000,000,000	Unija Alp / Uniya Alp
FFF,FFF,FFF,FFF,FFF	Zenja Zenjo Zen Alp, Zenja Zenjo Zen Kje, Zenja Zenjo Zen Ŭan, Zenja Zenjo Zen Sah, Zenja Zenjo Zen / Zenya Zenyo Zen Alp, Zenya Zenyo Zen Kye, Zenya Zenyo Zen Wan, Zenya Zenyo Zen Sah, Zenya Zenyo Zen
1,000,000,000,000,000	Uni Raj / Uni Rai
F,FFF,FFF,FFF,FFF,FFF	Zen Raj, Zenja Zenjo Zen Alp, Zenja Zenjo Zen Kje, Zenja Zenjo Zen Ŭan, Zenja Zenjo Zen Sah, Zenja Zenjo Zen / Zen Rai, Zenya Zenyo Zen Alp, Zenya Zenyo Zen Kye, Zenya Zenyo Zen Wan, Zenya Zenyo Zen Sah, Zenya Zenyo Zen
10,000,000,000,000,000	Unijo Raj / Uniyo Rai
FF,FFF,FFF,FFF,FFF,FFF	Zenjo Zen Raj, Zenja Zenjo Zen Alp, Zenja Zenjo Zen Kje, Zenja Zenjo Zen Ŭan, Zenja Zenjo Zen Sah, Zenja Zenjo Zen / Zenyo Zen Rai, Zenya Zenyo Zen Alp, Zenya Zenyo Zen Kye, Zenya Zenyo Zen Wan, Zenya Zenyo Zen Sah, Zenya Zenyo Zen
100,000,000,000,000,000	Unija Raj / Uniya Rai
FFF,FFF,FFF,FFF,FFF,FFF	Zenja Zenjo Zen Raj, Zenja Zenjo Zen Alp, Zenja Zenjo Zen Kje, Zenja Zenjo Zen Ŭan, Zenja Zenjo Zen Sah, Zenja Zenjo Zen / Zenya Zenyo Zen Rai, Zenya Zenyo Zen Alp, Zenya Zenyo Zen Kye, Zenya Zenyo Zen Wan, Zenya Zenyo Zen Sah, Zenya Zenyo Zen

6. Core Notation and Parsing Rules

- Every numeric glyph represents exactly one hexadecimal nibble (4 bits).
- The sixteen numeric glyphs map one-to-one to binary values 0000 through 1111.
- Numeric glyphs are not enclosed by an outer box.
- NULL, TRUE, and FALSE are special typed values and are not members of the numeric nibble set.
- HEAD and TAIL are optional framing markers. Their presence does not change the numerical or boolean value of the payload. These markers use existing characters and can be represented using standard character sets and fonts.
- Commas used for triad grouping are presentation separators and do not represent additional numeric digits.

- Spoken names are a pronunciation layer; changing a pronunciation must not change the underlying numeric mapping.
- An implementation must define its accepted character encoding, glyph version, and input direction before parsing graphical glyphs.

7. Implementation Considerations

Shodasha is intended to be implementable in digital and software environments without changing its underlying numeric representation. This section describes possible implementation approaches for fonts, input, parsing, and visual recognition. These considerations define practical requirements and possible directions rather than claiming that a complete implementation already exists.

7.1 Digital Font

A production font should contain the sixteen numeric glyphs and the five special characters. The glyph construction rules should be documented alongside the font so that alternate fonts remain recognizably compatible with the same notation.

7.2 Input and Keyboard

A keyboard implementation may map ordinary hexadecimal input 0-9 and A-F to Shodasha glyphs. This allows users to enter values without learning a new keyboard layout. A conversion layer can also transform existing hexadecimal strings into Shodasha for display.

7.3 Software Parser

A parser should represent the numeric glyphs internally as values from 0 to 15. Special characters should be represented as separate token types. HEAD and TAIL should be parsed as optional framing tokens rather than numeric digits.

7.4 OCR and Computer Vision

OCR should be treated as an implementation problem to be measured, not as an assumed property of the notation. A future dataset should contain multiple writers, handwriting speeds, pen types, rotations, scales, noise levels, and incomplete strokes. Recognition should be tested both with and without optional HEAD/TAIL framing.

7.5 Potential Applications

Shodasha may be useful for writing and displaying hexadecimal values in areas where compact visual representation is useful. Possible applications include computer networking addresses, memory addresses, microprocessor and integrated-circuit pin identification, digital colour values, byte values and binary data representation, hardware registers, and other systems that commonly use hexadecimal notation.

For example, hexadecimal values used in network addressing, memory locations, processor or circuit pin references, and colour representations could be displayed using Shodasha glyphs while retaining their original numerical values. This does not require changing the underlying data format or hardware representation; Shodasha would serve as an alternative human-facing notation.

These applications are potential use cases and should be evaluated through implementation and practical testing before any performance or usability advantage is claimed.

8. Validation Plan

The strongest next step for Shodasha is empirical testing. The current draft defines a design; the following experiments can establish whether the proposed advantages are supported in practice.

1. Visual Recognition
2. Writing Speed
3. Learning Curve
4. OCR Accuracy
5. Noise and Rotation
6. Transcription Accuracy
7. Parsing Accuracy
8. Font Consistency
9. Speech and Pronunciation

9. Known Limitations and Open Questions

- The current glyphs have not yet been validated through a large handwriting or OCR dataset.
- The proposal does not yet establish that Shodasha is faster to write than ordinary hexadecimal.
- The proposal does not establish that the glyphs are easier for humans to learn than 0-9/A-F.
- Optional HEAD/TAIL framing may improve explicit context, but its recognition benefit needs measurement.
- The spoken naming system is defined in detail in the draft, but its linguistic usability needs testing with speakers from different language backgrounds.
- Unicode standardization is not addressed by this draft. A Private-use Unicode code points or an application-specific encoding can be used during development.
- The system is a notation layer. It does not provide new arithmetic, new storage efficiency, or new processor instructions by itself.

10. Conclusion

Shodasha proposes a geometric way to write hexadecimal values using sixteen dedicated glyphs derived from four binary bits. Its strongest property is the deterministic relationship between the binary nibble and the geometry of the glyph. The special-character layer adds explicit forms for NULL and boolean values, while optional HEAD and TAIL markers can provide stream boundaries and orientation when needed.

The naming system extends the proposal beyond visual notation by providing a way to pronounce values without reading A-F as letters. Positional compounding and triad grouping are retained for larger values.

At this stage, Shodasha should be considered a draft notation/representation system rather than a proven replacement for hexadecimal. Its next stage should be implementation and measurement: a font, converter, parser, keyboard input method, handwriting dataset, OCR benchmark, and user study.

The purpose of this specification is not to claim that the problem has already been solved. It is to define the system clearly enough that other people can implement it, test it, criticize it, and improve it.